RECONMENDATION SYSTEM - 4

* MATRIX FACTORIZATION ⇒

Netflix

idea

Netflix

Why ?

$A =$	M_1	M_2	M_3	M_4	M_5
U_1	5	?	4	?	?
U_2	4	?	5	2	?
U_3	?	5	?	4	?
U_4	?	4	?	5	3

A = interaction matrix

A_{12}

(MATRIX FACTORIZATION)

(n, m)

Movie

A →

B

C

D

User 1

5 ✓

3 ✗

4 ✓

? ✗

User 2

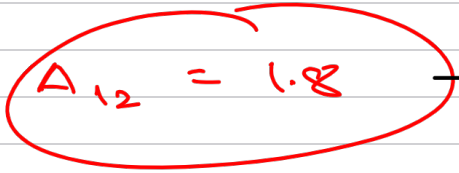
4 ✓

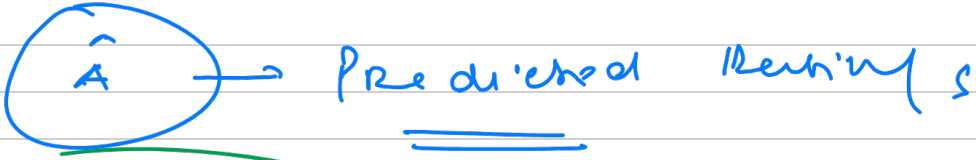
2 ✗

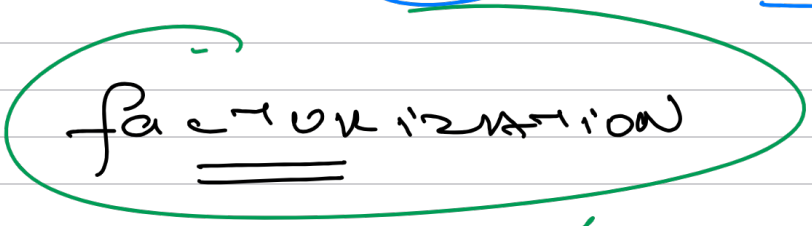
5 ✓

?





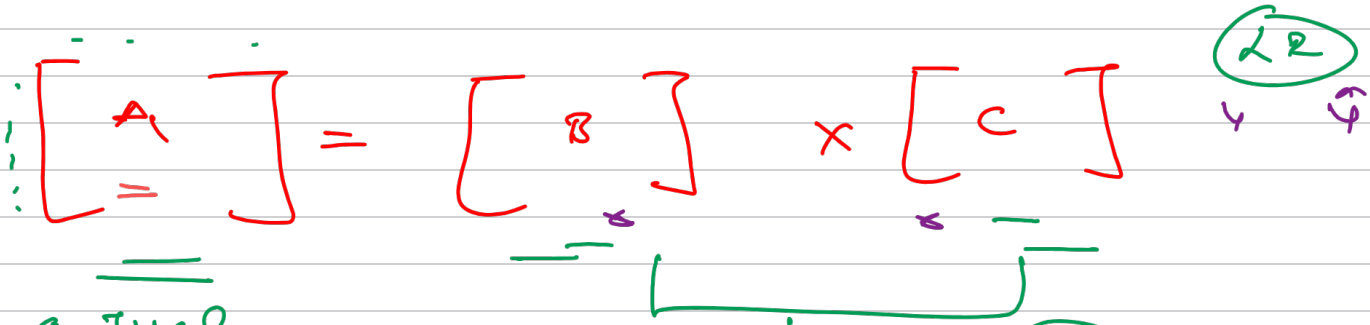




$6 = 2 \times 3$

$= = 1 \times 6$

MATRIX FACTORIZATION



$\hat{A} = A$ X

⇒ uncertainity

12 = 2 X 6

= 2 X 2 X 3

[A] →

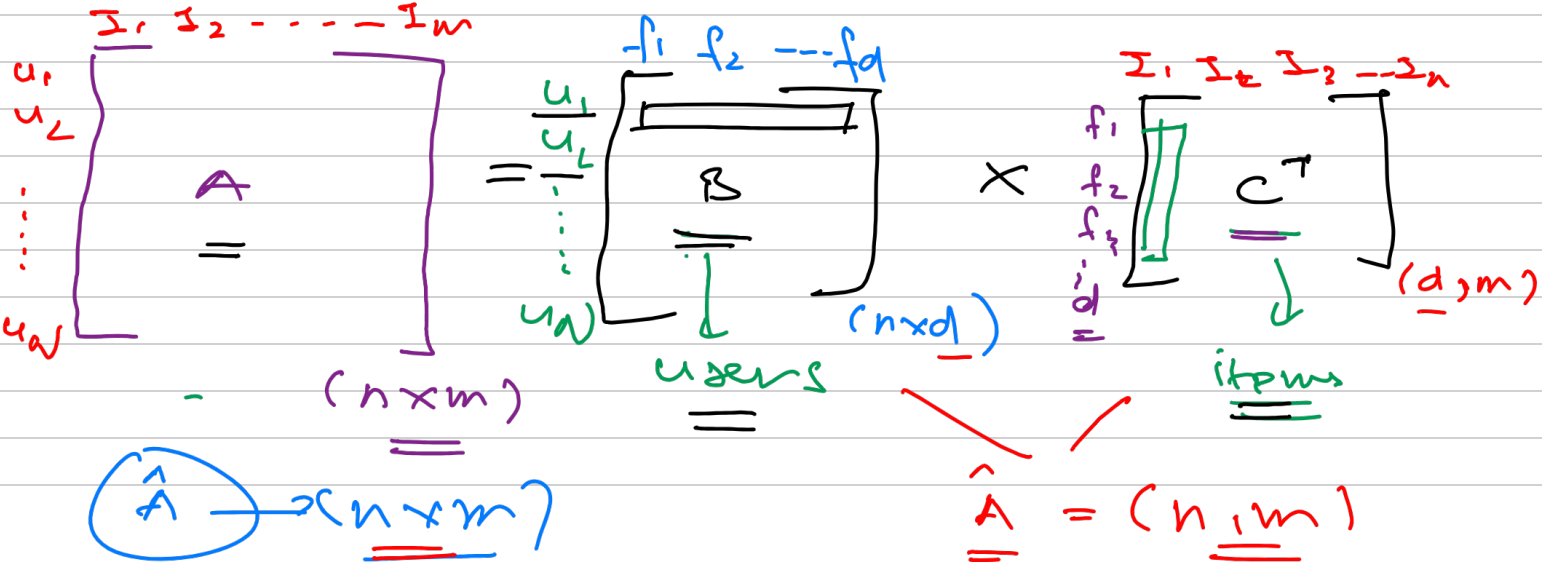
$A = B \times C \times D$

complexity

MATRIX factorization

↳ matrices

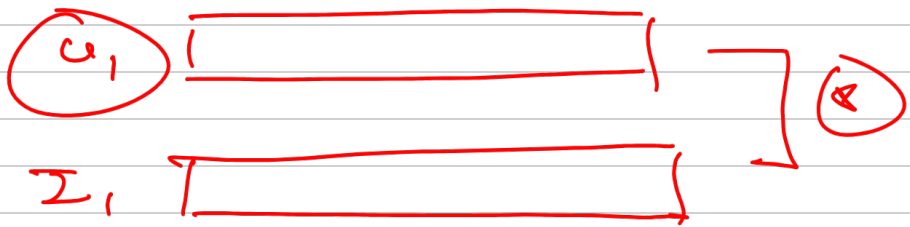
MF → Recommendation SYSTEM



$\hat{A} = B \cdot C^{T}$

(Predicted matrix)

CONTENT → (Creating features manually)



d =

1000 users 500 movies

d = 10 latent features

A → 1000 × 500

user

1000 × 10

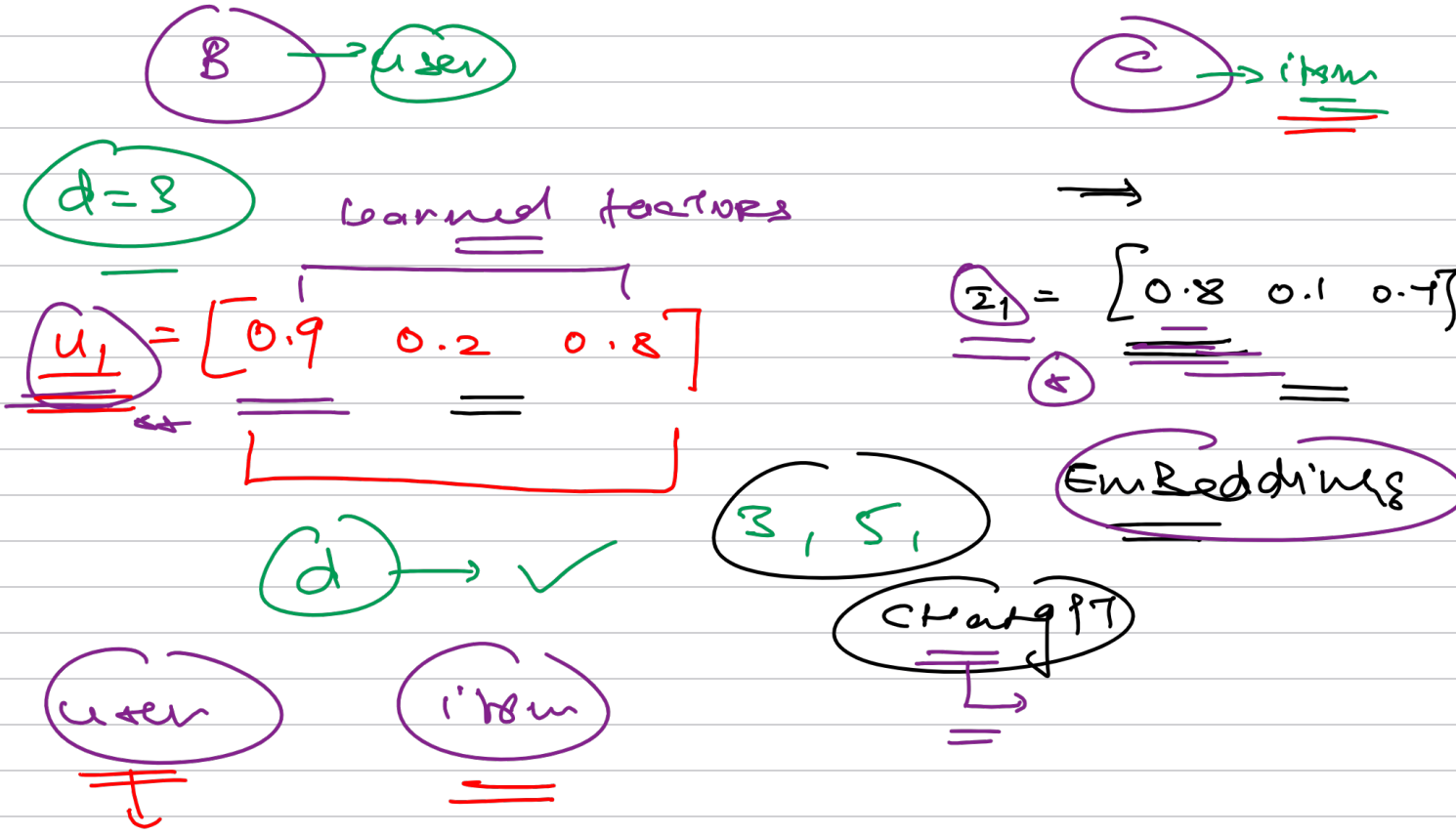
u →

item →

500 × 10

$\hat{A}$ = (users) (Items)^T

= (1000 × 10) (10 × 500)


B
user
C
items
d = 3
learned features
u_1 = [0.9 0.2 0.8]
z_1 = [0.8 0.1 0.7]
d
user
item
3, 5,
ChatGPT
Embeddings

(latent features)

$d = 3$

f_1 → ACTION

f_2 → Romance

f_3 → Sci fi

[ ] → Leaky

u_1 → [0.9 0.1 0.8]

I_1 = [0.4 0.1 0.95]

$u_1 I_1^T$

5 , 4 , 3

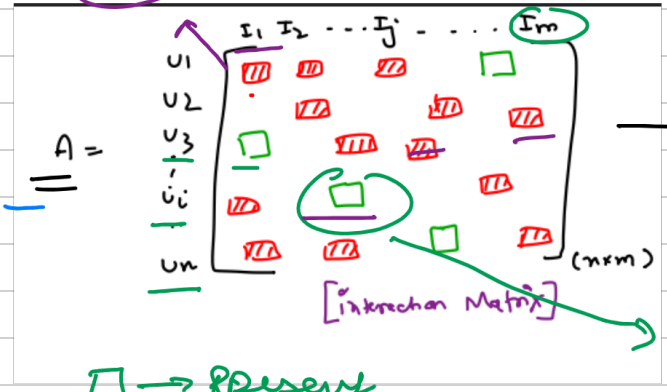
↑ Score / Rating

B → User

C → items

2

A_{11}



A =

Sparsity

NaN — 99.9

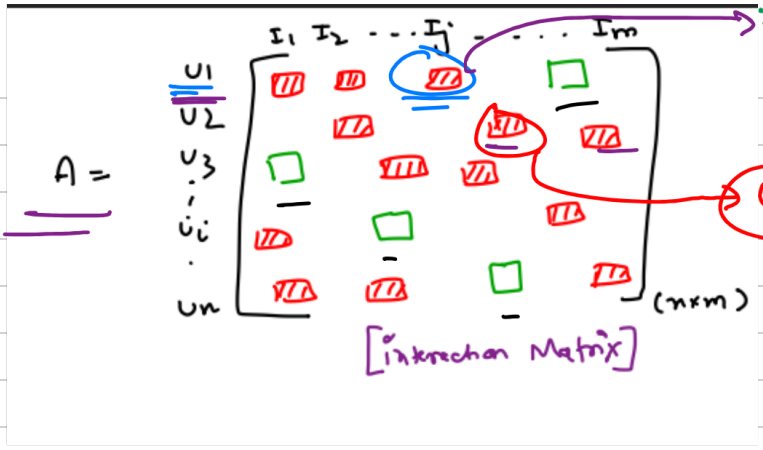
$A_{11'}$ = Pairing

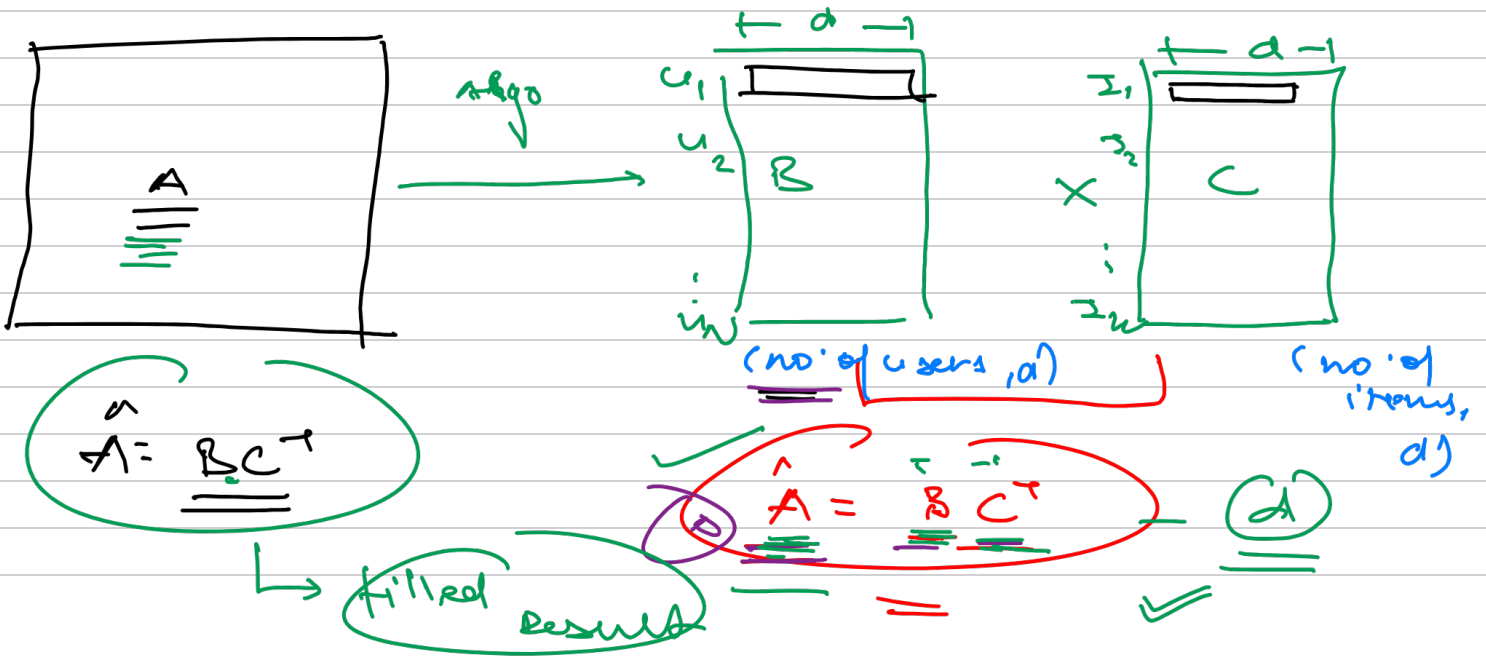
I → Present

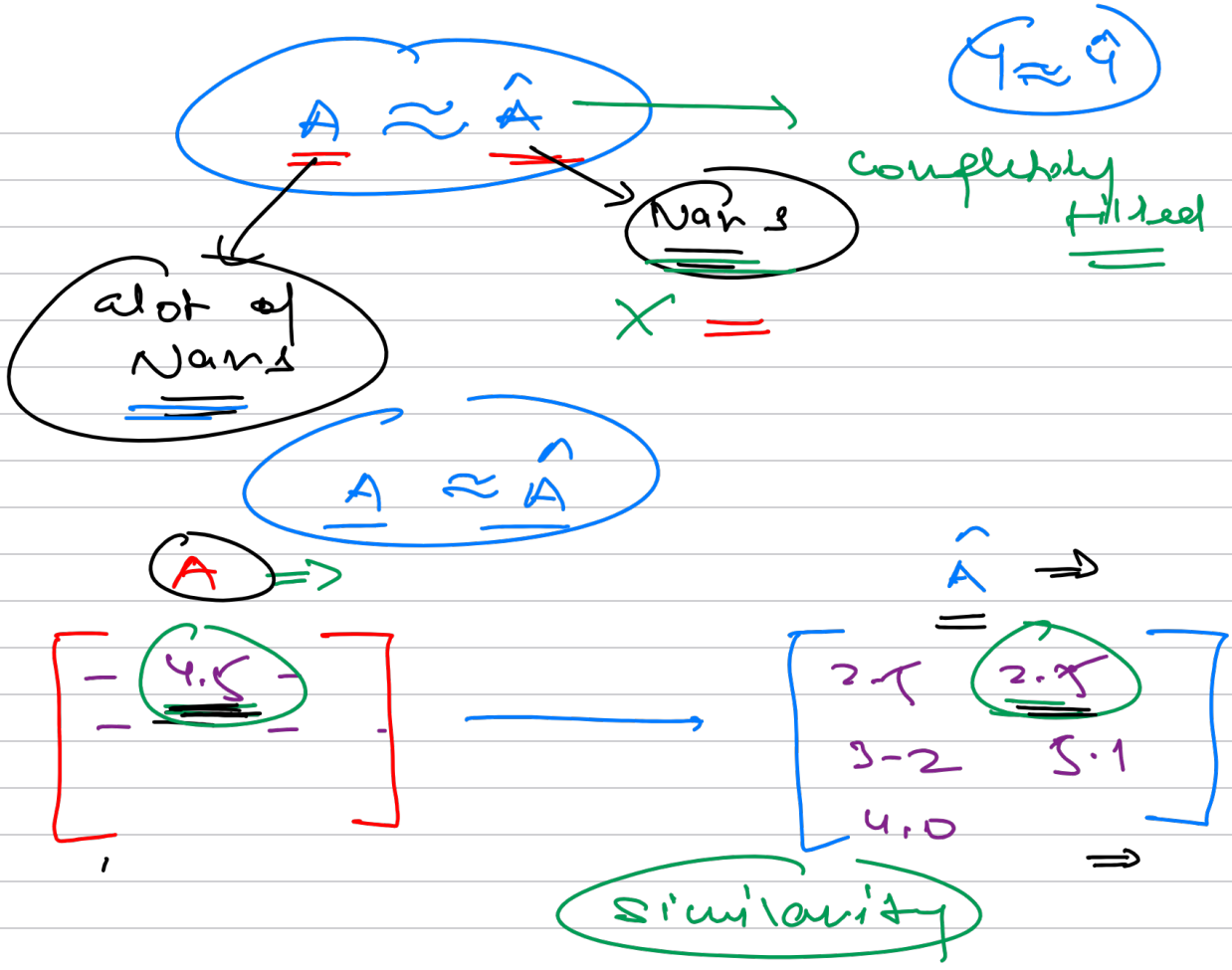
Im → N/A

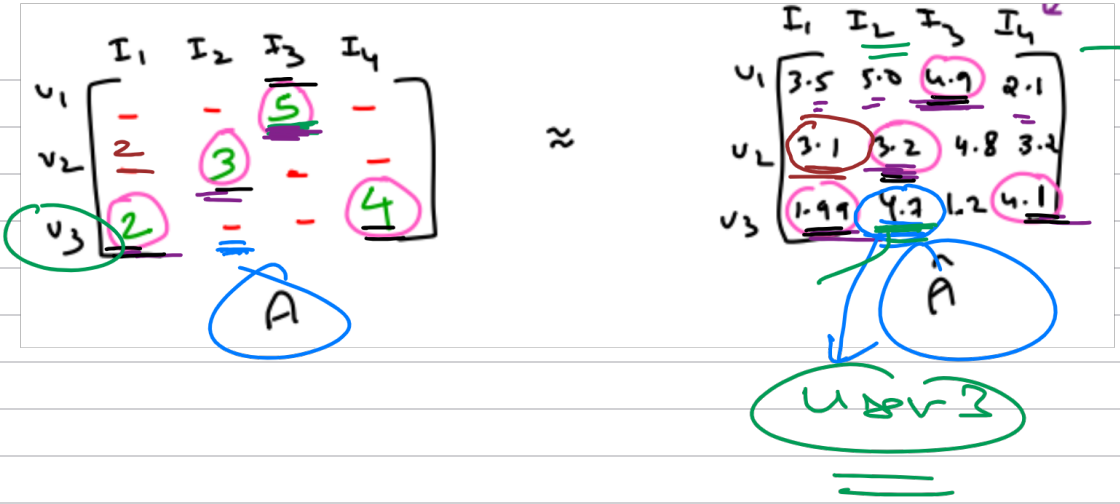
Recommendation Problem

MATRIX completion

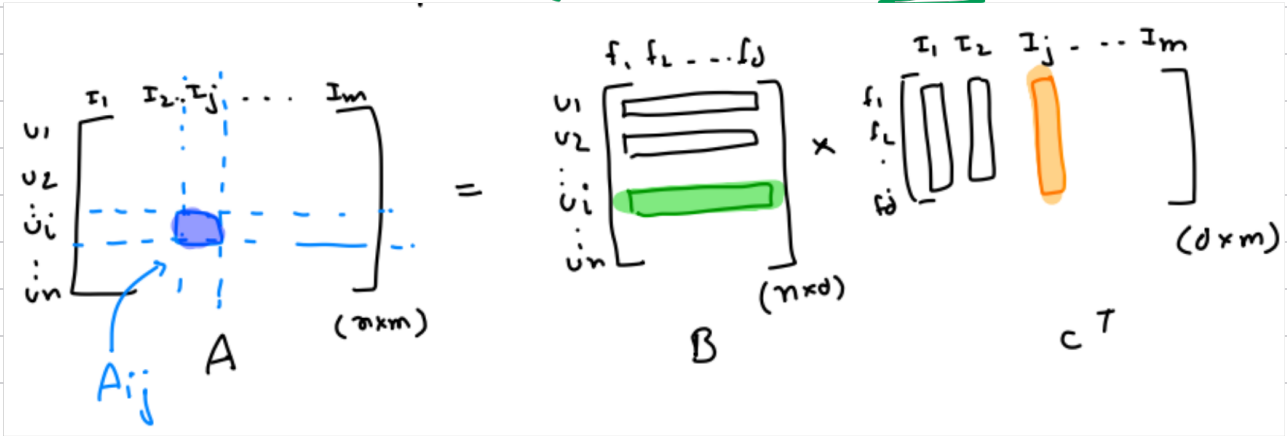

I1 I2 ... Ij ... Im
U1
U2
U3
Ui
Un
A =
(n×m)
[interaction Matrix]
2.75
NO
4.81

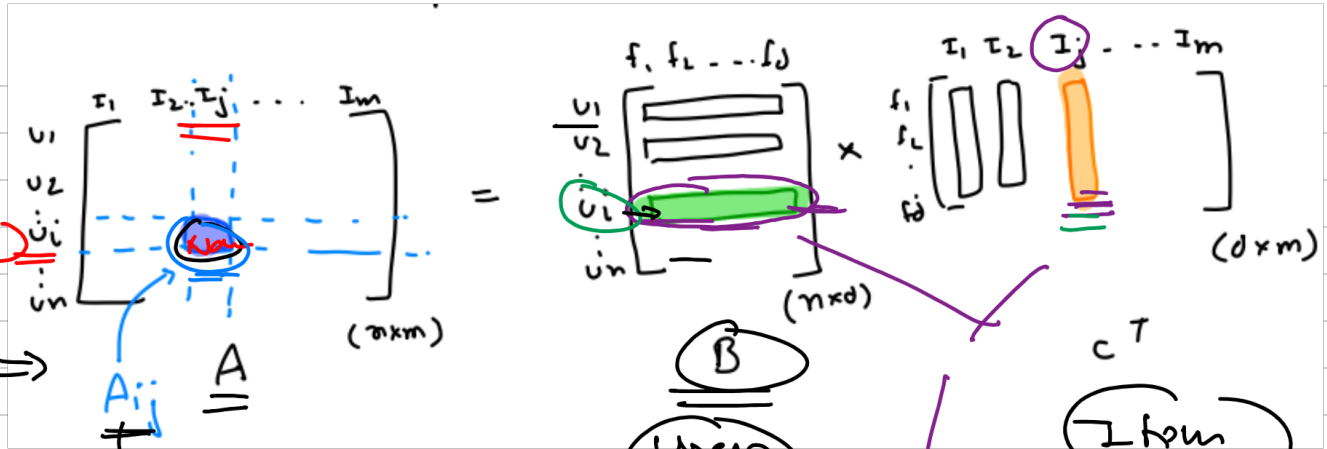

A
algo
d
B
C
U1
U2
Un
I1
I2
In
X
(no. of users, d)
(no. of items, d)
Â = BC^T
Â = BC^T
Filled Result


A ≈ Â
Y ≈ Ŷ
completely
filled
Nans
X
alot of
Nans
A ≈ Â
A
Â
4.5
2.7
3-2
5.1
4.0
similarity



Ⓐ A Rating $\approx$ $\hat{A}$ Ratings





$\hat{A} = B \cdot C^T$

$(n,d)\ (d,m)$

$\hat{A} = B \cdot C^T$

$A_{ij} = B_i^T \cdot C_j$

all

interaction

$(1 \times d)$

$(d \times 1)$

(1×1) → Rating $= A_{ij}$

2 matrices

B

(n, d)

C

(m, d)

(d, m)

(n, m)

individual vectors

vector multiply

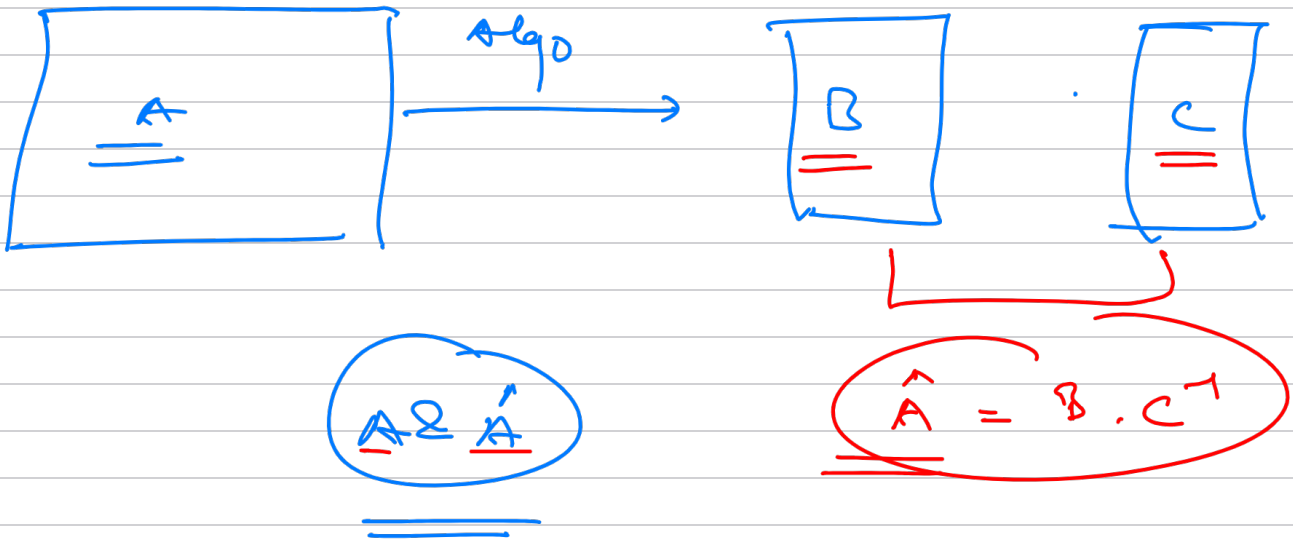
B_i^T $(d, 1)$. C_j^T $(d, 1)$

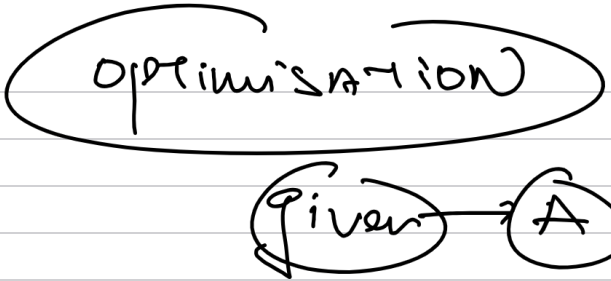
B_i^T $(1, d)$ × $(d, 1)$ $(1, d)$

optimisation

1 x 1 → call = A⁻¹

ex ⇒





To find → B & C

(n, a) (m, d)

$$\text{Loss} = \underset{B_i \; C_j}{\arg\min} \sum \left(A_{ij} - B_i^{+} C_j \right)^2$$

such that

A_{ij} is known in A

$\hat{A}_{ij}$

Optimisation → GD

in "Regression"

(1) Randomly init B & C

(h, d) (m, d)

(2) calculate loss [Sq. error] using existing values in A

(3) update B & C

GD

$\frac{\partial L}{\partial B}$, $\frac{\partial L}{\partial C}$

10:12 ⇒

Suppose:

$$A = \begin{bmatrix} 5 & ? & 4 \\ ? & 2 & ? \\ 3 & ? & 5 \end{bmatrix}$$

And:

$$\hat{A} = \begin{bmatrix} 4.8 & 4.2 & 4.1 \\ 3.7 & 2.2 & 1.9 \\ 3.1 & 4.5 & 4.8 \end{bmatrix}$$

MF

$A_{ij} = 5$

$\hat{A}_{ij} = 3.7$

$(5 - 3.7)^2 = 1.69$

Suppose we have:

	Interstellar	Titanic	Avengers
Rahul	5	1	5
Priya	1	5	?
Kashish	5	?	4

B C

f_1 f_2

Action

Romance

Rahul = [0.9 0.1] ⇒

Priya = [0.2, 0.9] ⇒

Kashish = [0.8 0.2] ⇒

Interstellar = [0.9 0.2]

Titanic = [0.1 0.95]

Avengers = [0.95 0.05]

$$[0.2 \quad 0.8] \begin{bmatrix} 0.95 \\ 0.05 \end{bmatrix}$$

$$= 0.09 + 0.045$$

$$= 0.235$$

Kashi's x Titanic

$$[0.8 \quad 0.2] \begin{bmatrix} 0.1 \\ 0.95 \end{bmatrix}$$

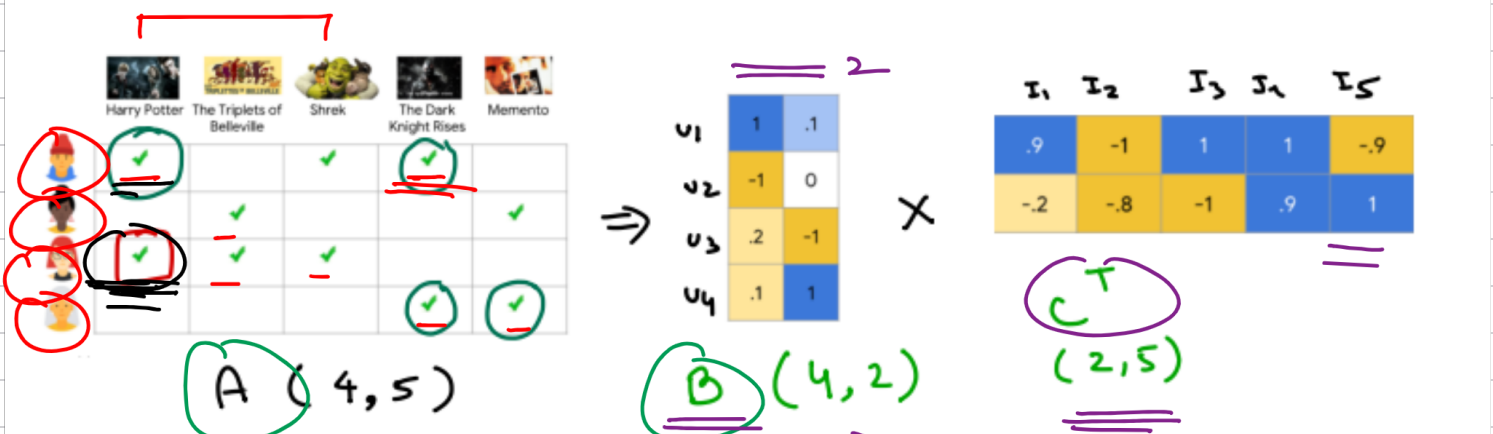
$$= 0.08 + 0.19 = 0.27$$

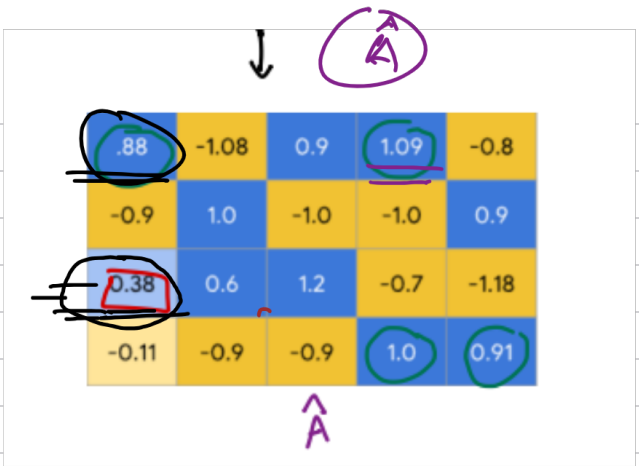
U_3 — I_{10}

$\hat{A}_{3,10} = U_3^T \cdot I_{10}$

Good

Actual ✗





$$\text{Loss} = \sum_{i,j} (A_{ij} - B_i^T C_j)^2$$

Rating existing

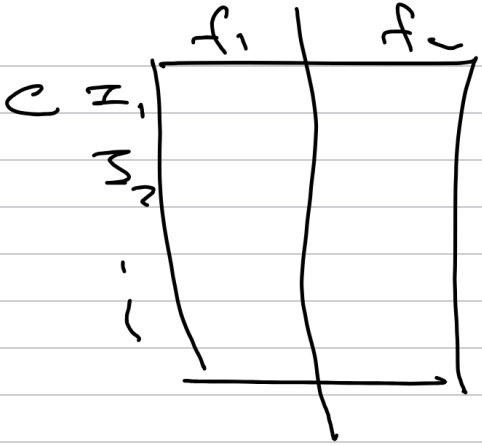
$\frac{\partial L}{\partial B_i}$ $\frac{\partial L}{\partial C_j}$

f_1 f_2 ... f_k

B (U) i

$B_{ik} = B_{ik} - \eta \frac{\partial L}{\partial B_{ik}}$

U fixed



$$\frac{\partial L}{\partial c_{jk}}$$

$$c_{jk} = c_{jk} - \eta \frac{\partial L}{\partial c_{jk}}$$

Repeat

init B & C

calculate error

Take derivatives

Update

$$L = \sum_{i,j} (A_{ij} - B_i^T C_j)^2$$

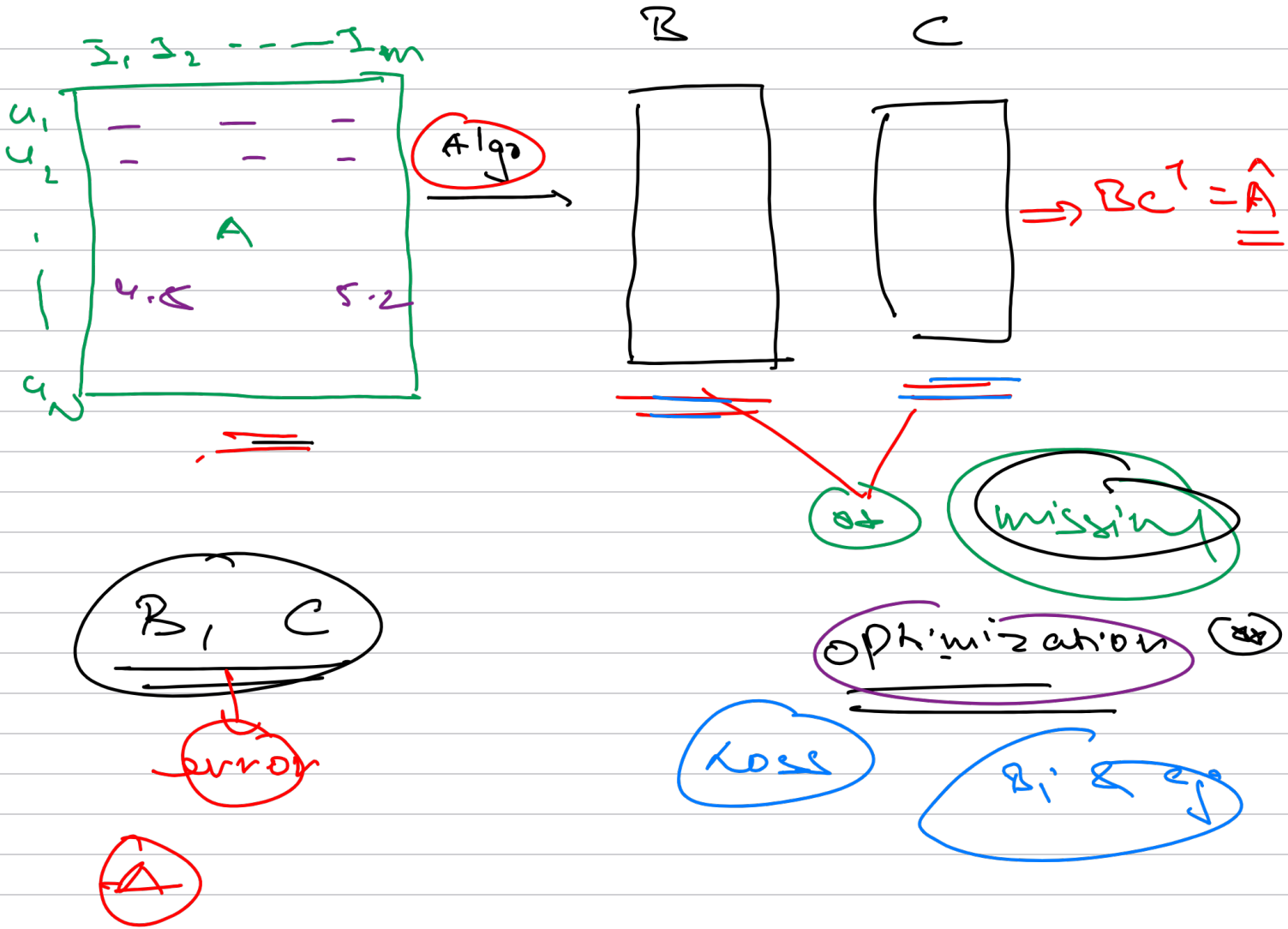
Rating available

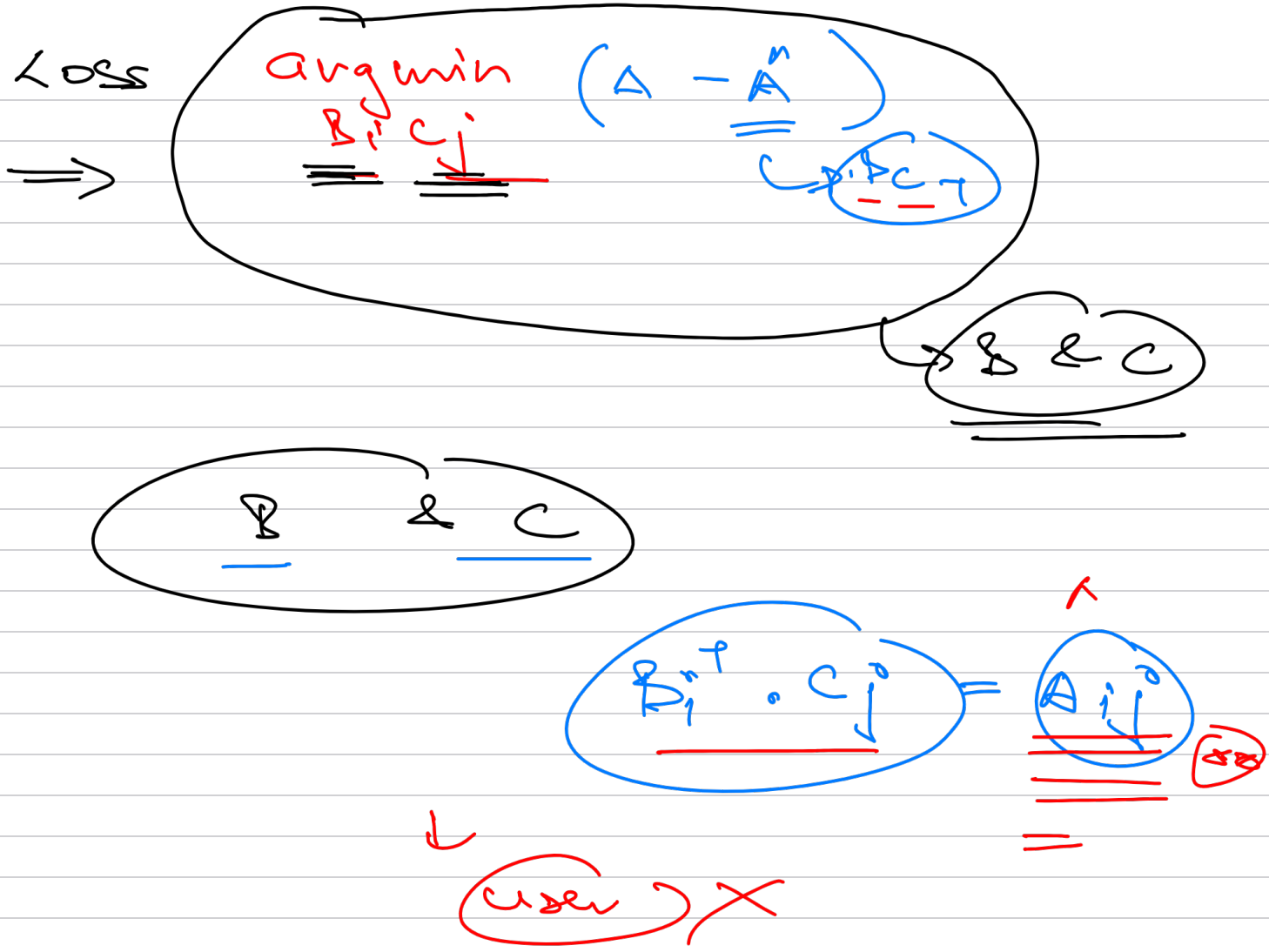
$$\frac{\partial L}{\partial B_i} = 2(A_{ij} - B_i^T C_j) \times \frac{\partial(-B_i^T C_j)}{\partial B_i}$$

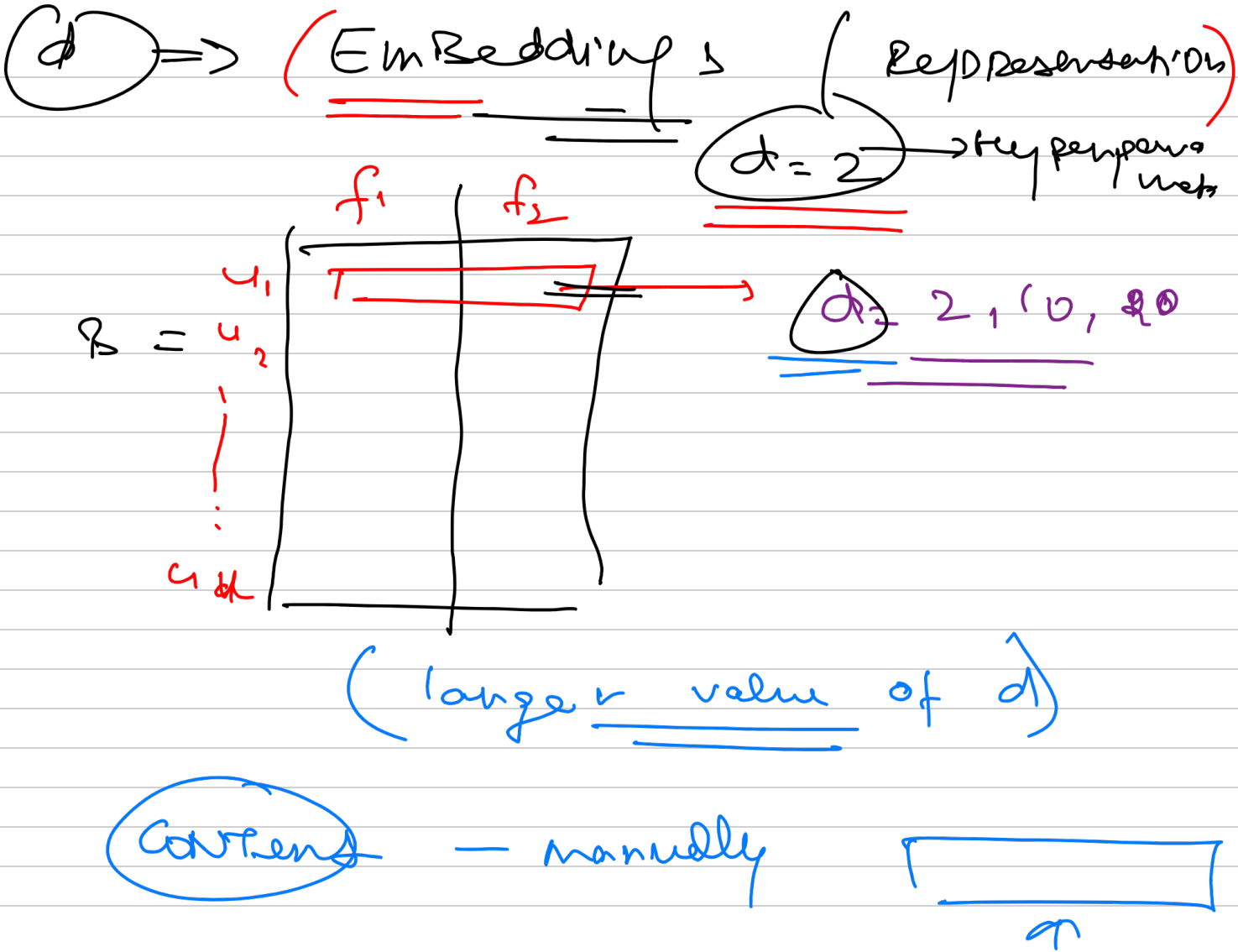
$$= 2(B_i^T C_j - A_{ij}) \times C_j$$

$$\boxed{\frac{\partial L}{\partial B_i} = 2(\hat{A}_{ij} - A_{ij}) \times C_j}$$

$$\frac{\partial L}{\partial C_{ij}} = 2\left(\hat{A}_{ij} - A_{ij}\right) \times B_i$$


I1 I2 ---- Im
u1
u2
un
A
4.6
5.2
Algo
B
C
⇒ BC^T = Â
os
missing
B, C
error
optimization
loss
Bi & Ci


Loss
argmin
B, C
(A - A^n)
B & C
B & C
B_i^T · C_j
A_ij
user

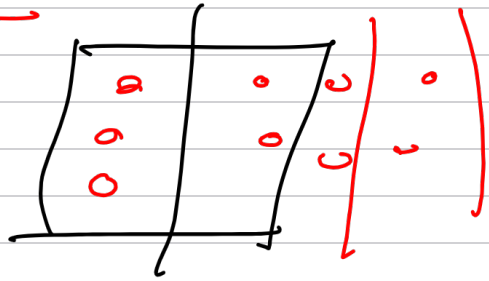

=> (Embedding) (Representations)
d = 2 → two component vectors
f1
f2
u1
u2
un
R =
T
(larger value of d)
Content — manually

d↑ =

↑ or ↓

↑ d → Better performance

d ↑ — —



d↑ → ↓

slower

Hyperparameter ↑ = complexity ↑

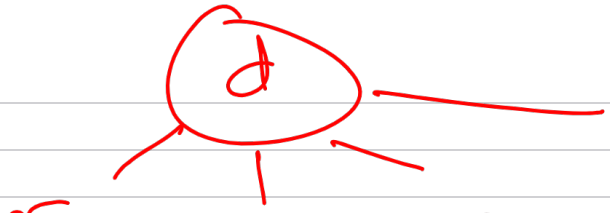
LR → ↑ = overfit

d↑ = overfit

d↓ = underfit

d = 2

y
1



HP Tuning

d = 2 → Huge error →

d = 5

d = 100

d = 500

Train

Val / Test

Stop

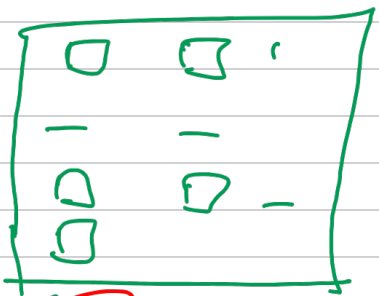
Where to stop



cl= Best value

GRID , CV

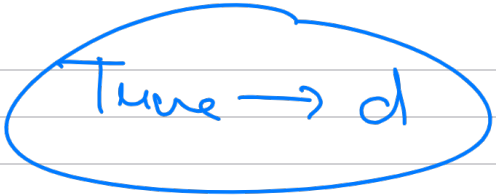
Train & Test split



(0.1 : 1)

80% → Training

20% → Testing



$d = 2$

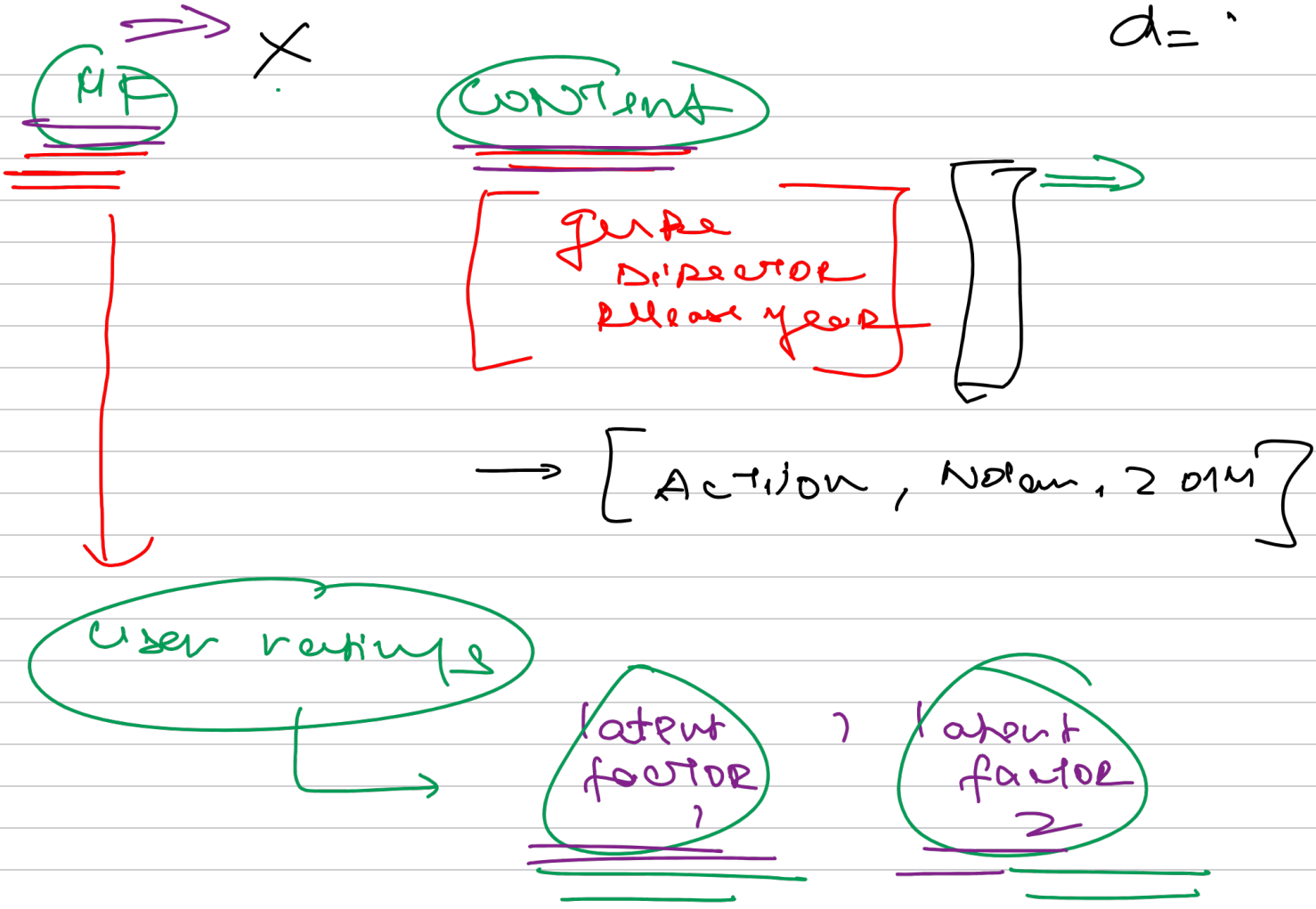
$d = 3$

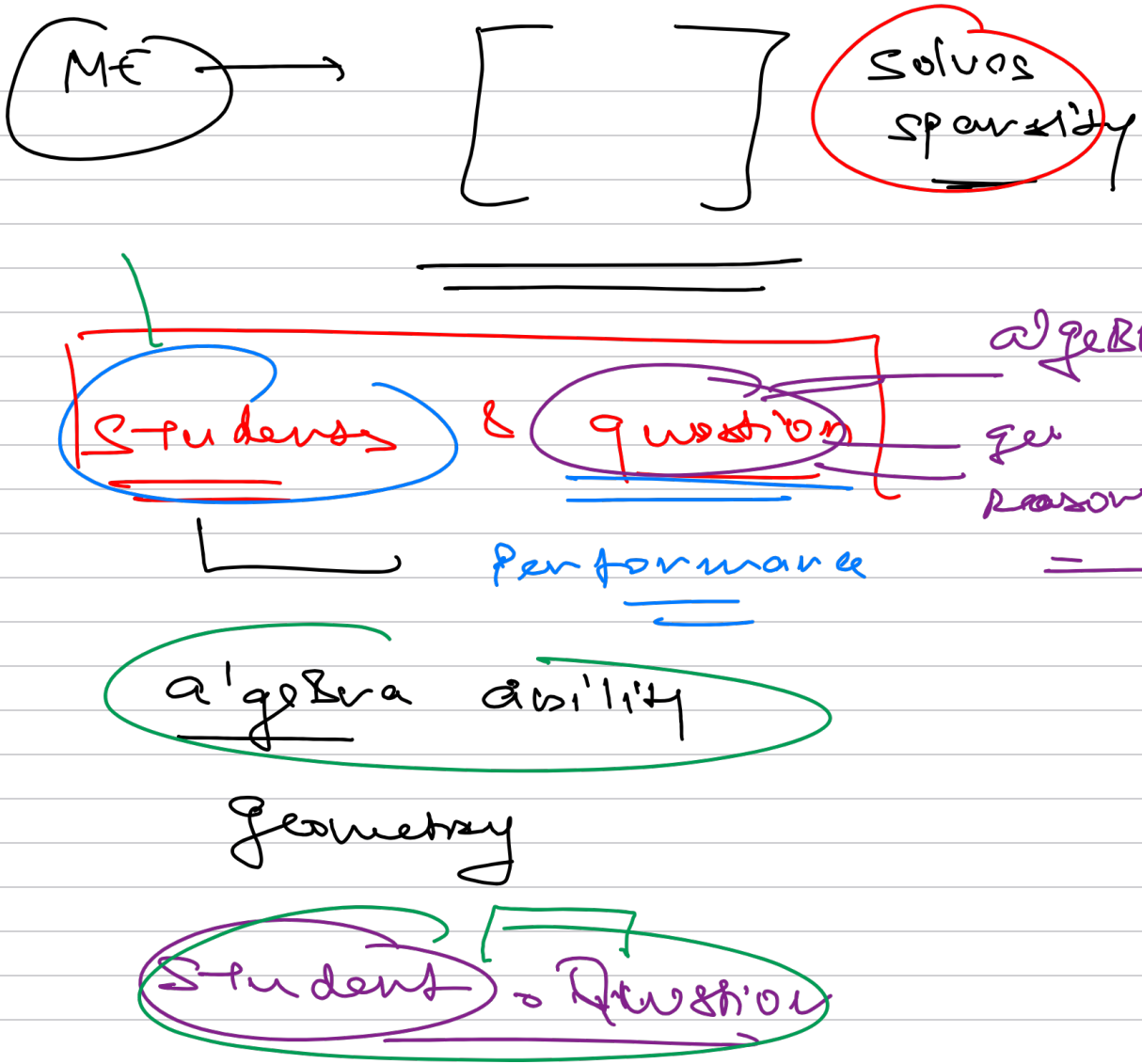
$d = 5$

$d = 10$ Train on

Test on

$d = 100$


MF
X
CONTENT
genre
Director
Release year
[Action, Nolan, 2014]
User ratings
latent factor 1
latent factor 2


ME
Solves
Students & Question
algebra
geometry
Performance
algebra ability
geometry
Student - Question